

Java control statements

30 September 2026 06:33

Java runtime environment (JVM) executes the code from top to bottom. The statements in the code are executed according to the order in which they appear. However, Java provides statements that can be used to control the flow of Java code. Such statements are called control flow statements. It is one of the fundamental features of Java which provides a smooth flow of program.

Types of Control Flow Statements

Java provides three types of control flow statements.

1. Decision making statements.

- i. if statements
- ii. switch statements

2. Loop statements

- i. do...while loop
- ii. while loop
- iii. for loop
- iv. for-each loop

3. Jump statements

- i. break
- ii. continue

Java Decision-Making Statements

As the name suggests, decision-making statements decide which statement to execute and when. Decision-making statements evaluate the boolean expression and control the program flow depending upon the result of the condition provided. There are two types of decision-making statements in Java. That is 'if' statement and 'switch' statement.

'if' Statement

Syntax:

```
if(condition){  
    //if true execute the block  
}
```

'if...else' statement

Syntax:

```
if(condition){
    //if true execute the block
}
else{
    //if false execute the block
}
```

'if...else if' statement

Syntax:

```
if(condition){
    //if true execute the block
}
else if{
    //if false execute the block
}
else if{
    //if false execute the block
}
else if{
    //if false execute the block
}
...
else{
    //if all the 'if' block fails to execute
    //this block will execute, we call it dangling else
}
```

'switch' Statement

In Java, switch statements are similar to if - else - if statements. The switch statement contains multiple blocks of code called cases and a single case is executed based on the variable which is being switched. The switch statement is easier to use instead of if - else - if statement. It also enhances the readability of the program.

Points to be noted about switch statement:

- The case variable can be int, short, byte, char or enumeration. String type is also supported since version 7 of Java.
- Cases cannot be duplicated.
- Default statement is executed when any of the case doesn't match the value of the expression. It is optional.

- Break statement terminates the switch block when the condition is satisfied. It is optional if not used next case is executed.
- While using switch statements, we must notice that the case expression will be the same type as the variable. However, it also be a constant value.

Syntax:

```
switch(expression){
    case value1:
        Statement(s)
        break;
    case value2:
        Statement(s)
        break;
    case value3:
        Statement(s)
        break;
    case value4:
        Statement(s)
        break;
    ...
    default:
        Statement(s)
```

Data-Input through keyboard

```
Scanner sc = new Scanner(System.in);
```

Scanner class is present in java.util package, so we need to import the java.util package to create the object of Scanner class.

Scanner class contains methods for data input in Java.

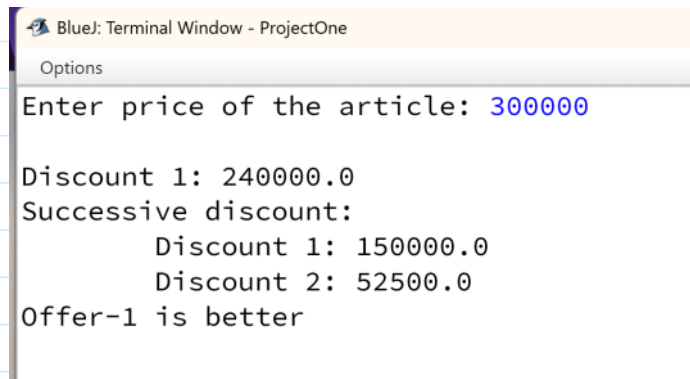
Data type	Method from Scanner class
boolean	nextBoolean()
byte	nextByte()
short	nextShort()
int	nextInt()
long	nextLong()
float	nextFloat()
double	nextDouble()

String	next() =>it take a word nextLine() => you can input a sentence
char	next().charAt(0)

```
package Chapter2;
import java.util.*;
```

```
/**
 * Write a description of class Q4 here.
 * A shopkeeper offers 80% discount on purchasing articles whereas
 * the other shopkeeper offers two successive discount 60% and 25%
 * for purchasing the same articles.
 * Write a program in Java to compute and display which is better
 * offer for a customer. Take the price of an article as an input.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Q4
{
    public static void main(String args[]){
        System.out.print("\f"); //clears terminal window
        Scanner sc = new Scanner(System.in);
        float price, dis1, dis2, dis3;
        System.out.print("Enter price of the article: ");
        price = sc.nextFloat();
        dis1 = price*0.8f;
        dis2 = price*0.5f;
        dis3 = (price-dis2)*0.35f;
        System.out.print("\nDiscount 1: " + dis1);
        System.out.print("\nSuccessive discount: ");
        System.out.print("\n\tDiscount 1: " + dis2);
        System.out.print("\n\tDiscount 2: " + dis3);
        if(dis1 > (dis2+dis3)){
            System.out.print("\nOffer-1 is better");
        }
        else{
            System.out.print("\nOffer-2 is better");
        }
    }
}
```

Output:



A screenshot of a BlueJ terminal window titled "BlueJ: Terminal Window - ProjectOne". The window has a menu bar with "Options". The output text is as follows:

```
Enter price of the article: 300000

Discount 1: 240000.0
Successive discount:
    Discount 1: 150000.0
    Discount 2: 52500.0
Offer-1 is better
```